

ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES
BENI MELLAL

Département Génie Logiciel

RAPPORT DE TRAVAUX PRATIQUES

TP 01 : Programmation Orientée Objet
Langage Java

Exercice 1.1 : Gestion d'animaux dans un Zoo
Exercice 1.2 : Gestion des Véhicules de Transport
Exercice 1.3 : Gestion de Comptes Bancaires

Encadré par : Pr. BE.ELBAGHAZAOUI
Année universitaire 2025 - 2026

Table des Matières

1. Introduction
2. Rappel des concepts POO
3. Exercice 1.1 – Gestion d'animaux dans un Zoo
 - 3.1 Présentation du problème
 - 3.2 Structure des classes
 - 3.3 Code complété
 - 3.4 Réponses aux questions théoriques
 - 3.5 Résultat d'exécution
4. Exercice 1.2 – Gestion des Véhicules de Transport
 - 4.1 Présentation du problème
 - 4.2 Structure des classes
 - 4.3 Code complété
 - 4.4 Réponses aux questions théoriques
 - 4.5 Résultat d'exécution
5. Exercice 1.3 – Gestion de Comptes Bancaires
 - 5.1 Présentation du problème
 - 5.2 Implémentation complète
 - 5.3 Résultat d'exécution
6. Synthèse des concepts POO dans les exercices
7. Conclusion

1. Introduction

Ce rapport présente le travail réalisé dans le cadre du TP 01 du module Génie Logiciel, encadré par le Pr. BE.ELBAGHAZAOUI à l'École Nationale des Sciences Appliquées de Beni Mellal (ENSA-BM). Ce TP est consacré à la Programmation Orientée Objet (POO) en Java, un paradigme fondamental dans le développement logiciel moderne.

Le TP se compose de trois exercices progressifs, chacun mettant en pratique les quatre piliers de la POO : l'encapsulation, l'héritage, l'abstraction et le polymorphisme. Le premier exercice porte sur la modélisation d'un système de gestion d'animaux dans un zoo. Le deuxième traite de la gestion de véhicules pour une société de transport. Le troisième exercice propose la création d'un système de gestion de comptes bancaires.

L'objectif pédagogique est de consolider la compréhension théorique de ces concepts tout en développant la capacité à les appliquer concrètement dans des programmes Java fonctionnels.

2. Rappel des concepts POO

Avant d'aborder les exercices, il est utile de rappeler les quatre piliers fondamentaux de la Programmation Orientée Objet :

Concept	Définition
Encapsulation	Principe qui consiste à regrouper les données (attributs) et les méthodes qui les manipulent dans une même unité (classe), tout en restreignant l'accès direct aux données internes via des modificateurs d'accès (private, protected, public).
Héritage	Mécanisme permettant à une classe (sous-classe) de réutiliser les attributs et méthodes d'une autre classe (superclasse), établissant une relation « est-un ». En Java, on utilise le mot-clé extends.
Abstraction	Processus qui consiste à exposer uniquement les caractéristiques essentielles d'un objet tout en cachant les détails d'implémentation. En Java, cela se fait via les classes abstraites (abstract class) et les interfaces.
Polymorphisme	Capacité d'un même appel de méthode à produire des comportements différents selon le type réel de l'objet. En Java, cela se manifeste par la redéfinition de méthodes (@Override) dans les sous-classes.

3. Exercice 1.1 – Gestion d'animaux dans un Zoo

3.1 Présentation du problème

On souhaite modéliser un système de gestion d'animaux dans un zoo. Chaque animal possède un nom et une méthode pour produire un son spécifique. Les animaux sont classés en deux catégories : les mammifères et les oiseaux. L'objectif est d'utiliser les concepts POO pour concevoir une hiérarchie de classes flexible et extensible.

3.2 Structure des classes

Le diagramme de classes se présente comme suit :

Animal (abstraite)	Mammifere	Oiseau
- nom : String (private)	extends Animal	extends Animal
+ getNom() : String	+ faireDuBruit() : void	+ faireDuBruit() : void
+ faireDuBruit() : abstract	→ affiche "grogne"	+ voler() : void

3.3 Code complété

Classe abstraite Animal

```
abstract class Animal {
    private String nom;

    public Animal(String nom) {
        this.nom = nom;
    }

    public String getNom() {
        return nom;
    }

    public abstract void faireDuBruit();
}
```

Classe Mammifere

```
class Mammifere extends Animal {
    public Mammifere(String nom) {
        super(nom);
    }

    @Override
    public void faireDuBruit() {
        System.out.println(getNom() + " grogne.");
    }
}
```

Classe Oiseau (avec méthode voler() complétée)

```
class Oiseau extends Animal {
    public Oiseau(String nom) {
        super(nom);
    }

    @Override
    public void faireDuBruit() {
        System.out.println(getNom() + " chante.");
    }

    // Question 5 : Méthode voler() complétée
    public void voler() {
        System.out.println("L'oiseau vole.");
    }
}
```

Classe principale Zoo (avec appel à voler())

```
public class Zoo {
    public static void main(String[] args) {
        Animal tigre = new Mammifere("Tigre");
        Animal perroquet = new Oiseau("Perroquet");

        tigre.faireDuBruit();
        perroquet.faireDuBruit();

        // Question 6 : cast vers Oiseau pour voler()
        Oiseau perroquetOiseau = (Oiseau) perroquet;
        perroquetOiseau.voler();
    }
}
```

3.4 Réponses aux questions théoriques

Partie 1 : Définitions théoriques

Q1 – Encapsulation : L'encapsulation est le principe qui consiste à cacher les données internes d'un objet et à n'exposer que les méthodes nécessaires pour y accéder. Dans la classe Animal, l'attribut nom est déclaré private, ce qui interdit tout accès direct depuis l'extérieur. Seule la méthode publique getNom() permet de récupérer sa valeur. Cela protège l'intégrité des données.

Q2 – Héritage : L'héritage est un mécanisme qui permet à une classe enfant de réutiliser les attributs et méthodes d'une classe parent. Dans ce code, les classes Mammifere et Oiseau héritent de la classe Animal grâce au mot-clé extends. Elles héritent de l'attribut nom et de la méthode getNom(), tout en fournissant leur propre implémentation de faireDuBruit().

Q3 – Abstraction : L'abstraction consiste à définir un contrat (interface) sans fournir l'implémentation concrète. Dans cet exercice, la classe Animal est déclarée abstract, ce qui signifie qu'on ne peut pas l'instancier directement. La méthode faireDuBruit() est également abstraite : chaque sous-classe est obligée de fournir sa propre version.

Q4 – Polymorphisme : Le polymorphisme permet à un même appel de méthode de produire des comportements différents selon le type réel de l'objet. Dans la méthode main(), les variables tigre et perroquet sont déclarées de type Animal, mais l'appel faireDuBruit() exécute respectivement la version de Mammifere ("grogne") et celle de Oiseau ("chante"). C'est le polymorphisme par redéfinition.

Partie 2 : Complétion du code

Q5 : La méthode voler() a été ajoutée dans la classe Oiseau. Elle affiche "L'oiseau vole." lorsqu'elle est appelée.

Q6 : Pour appeler voler() sur l'objet perroquet (déclaré comme Animal), il faut effectuer un cast explicite vers le type Oiseau : Oiseau perroquetOiseau = (Oiseau) perroquet; puis appeler perroquetOiseau.voler();

Partie 3 : Identification des mots-clés

Concept	Mot-clé Java
Héritage	extends
Redéfinition de méthode	@Override
Abstraction	abstract (classe et méthode)

3.5 Résultat d'exécution

```
Tigre grogne.
Perroquet chante.
L'oiseau vole.
```


4. Exercice 1.2 – Gestion des Véhicules de Transport

4.1 Présentation du problème

On souhaite implémenter un système de gestion de véhicules pour une société de transport. Il existe différents types de véhicules (voitures, camions) qui partagent des caractéristiques communes (marque, modèle, année) mais ont des comportements spécifiques. On doit également ajouter une classe Moto et une méthode demarrer().

4.2 Structure des classes

Vehicule (abstraite)	Voiture	Camion	Moto (ajoutée)
- marque, modele, annee	- nombreDePortes : int	- capaciteDeCharge : double	- typeDeGuidon : String
+ getters()	+ afficherDetails()	+ afficherDetails()	+ afficherDetails()
+ demarrer() (ajoutée)	appelle demarrer()	appelle demarrer()	appelle demarrer()

4.3 Code complété

Méthode demarrer() ajoutée dans Vehicule (Q5)

```
public void demarrer() {
    System.out.println("Le véhicule démarre.");
}
```

Cette méthode est appelée dans afficherDetails() de chaque sous-classe avant l'affichage des détails spécifiques.

Nouvelle classe Moto (Q6)

```
class Moto extends Vehicule {
    private String typeDeGuidon;

    public Moto(String marque, String modele,
                int annee, String typeDeGuidon) {
        super(marque, modele, annee);
        this.typeDeGuidon = typeDeGuidon;
    }

    @Override
    public void afficherDetails() {
        demarrer();
        System.out.println("Moto: " + getMarque()
            + " " + getModele() + " ("
            + getAnnee() + "), Guidon: "
            + typeDeGuidon);
    }
}
```

Classe principale modifiée


```
public class GestionTransport {  
    public static void main(String[] args) {  
        Vehicule v1 = new Voiture("Toyota", "Corolla", 2021, 4);  
        Vehicule v2 = new Camion("Volvo", "FMX", 2019, 12.5);  
        Vehicule v3 = new Moto("Yamaha", "MT-07", 2023, "Sport");  
  
        v1.afficherDetails();  
        v2.afficherDetails();  
        v3.afficherDetails();  
    }  
}
```

4.4 Réponses aux questions théoriques

Q1 – Encapsulation : Les attributs marque, modele et annee sont déclarés private dans la classe Vehicule. Leur accès se fait uniquement via les getters publics getMarque(), getModele() et getAnnee(). De même, nombreDePortes dans Voiture et capaciteDeCharge dans Camion sont privés.

Q2 – Héritage : Les classes Voiture, Camion et Moto héritent toutes de Vehicule via le mot-clé extends. Elles réutilisent les attributs communs et le constructeur parent via super(). Chaque sous-classe ajoute ses propres attributs spécifiques.

Q3 – Polymorphisme : Dans la méthode main(), les trois objets sont déclarés de type Vehicule, mais l'appel à afficherDetails() produit un résultat différent pour chaque type réel (Voiture, Camion ou Moto). C'est la liaison dynamique qui détermine quelle version de la méthode est exécutée.

Q4 – Abstraction : La classe Vehicule est abstraite car un "véhicule" générique est trop vague pour être instancié. La méthode abstraite afficherDetails() impose à chaque sous-classe de fournir sa propre implémentation avec les détails spécifiques.

Mots-clés identifiés (Q7)

Concept	Mot-clé Java
Redéfinition de méthode	@Override
Héritage	extends
Appel constructeur parent	super()

4.5 Résultat d'exécution

```

Le véhicule démarre.
Voiture: Toyota Corolla (2021), Portes: 4

Le véhicule démarre.
Camion: Volvo FMX (2019), Capacité: 12.5 tonnes

Le véhicule démarre.
Moto: Yamaha MT-07 (2023), Guidon: Sport

```

5. Exercice 1.3 – Gestion de Comptes Bancaires

5.1 Présentation du problème

On doit créer un programme qui gère différents types de comptes bancaires : CompteCourant et CompteEpargne. Chaque compte possède un numéro et un solde, avec des méthodes pour déposer et retirer de l'argent. Le CompteEpargne ajoute une méthode pour calculer les intérêts.

5.2 Implémentation complète

Classe de base : CompteBancaire

```
class CompteBancaire {
    private String numeroCompte;
    private double solde;

    public CompteBancaire(String numero, double solde) {
        this.numeroCompte = numero;
        this.solde = solde;
    }

    public void deposter(double montant) {
        if (montant > 0) {
            solde += montant;
            System.out.println("Dépôt de "
                + montant + " € effectué.");
        }
    }

    public void retirer(double montant) {
        if (montant > solde) {
            System.out.println("Solde insuffisant.");
        } else {
            solde -= montant;
            System.out.println("Retrait de "
                + montant + " € effectué.");
        }
    }

    public void afficherInfos() {
        System.out.println("Compte " + numeroCompte
            + " | Solde: " + solde + " €");
    }
}
```

Classe dérivée : CompteEpargne

```
class CompteEpargne extends CompteBancaire {
    public CompteEpargne(String numero, double solde) {
        super(numero, solde);
    }

    public void calculerInterets(double taux) {
        double interets = getSolde() * taux / 100;
        deposter(interets);
    }
}
```

```

        System.out.println("Intérêts: "
            + interets + " € (taux " + taux + "%)");
    }
}

```

Classe principale : Banque

```

public class Banque {
    public static void main(String[] args) {
        // Compte Courant
        CompteCourant cc = new CompteCourant("CC-001", 1000);
        cc.deposer(500);
        cc.retirer(200);
        cc.afficherInfos();

        // Compte Épargne
        CompteEpargne ce = new CompteEpargne("CE-001", 5000);
        ce.deposer(1000);
        ce.calculerInterets(3.5);
        ce.afficherInfos();
    }
}

```

5.3 Résultat d'exécution

```

--- Compte Courant ---
Dépôt de 500.0 € effectué. Nouveau solde : 1500.0 €
Retrait de 200.0 € effectué. Nouveau solde : 1300.0 €
Compte CC-001 | Solde : 1300.0 €

--- Compte Épargne ---
Dépôt de 1000.0 € effectué. Nouveau solde : 6000.0 €
Dépôt de 210.0 € effectué. Nouveau solde : 6210.0 €
Intérêts: 210.0 € (taux 3.5%)
Compte CE-001 | Solde : 6210.0 €

```

6. Synthèse des concepts POO dans les exercices

Le tableau suivant récapitule où et comment chaque concept POO est appliqué dans les trois exercices :

Concept	Ex. 1.1 (Zoo)	Ex. 1.2 (Véhicules)	Ex. 1.3 (Banque)
Encapsulation	nom (private) + getNom()	marque, modele, annee (private) + getters	numeroCompte, solde (private) + getters
Héritage	Mammifere, Oiseau extends Animal	Voiture, Camion, Moto extends Vehicule	CompteEpargne extends CompteBancaire
Abstraction	Animal abstraite, faireDuBruit() abstraite	Vehicule abstraite, afficherDetails() abstraite	Classe de base concrète (pas d'abstraction explicite)
Polymorphisme	faireDuBruit() selon le type réel	afficherDetails() selon le type réel	deposer() et retirer() hérités et réutilisés

7. Conclusion

Ce TP nous a permis de mettre en pratique de manière concrète les quatre piliers fondamentaux de la Programmation Orientée Objet en Java. À travers trois exercices progressifs, nous avons pu constater comment ces concepts se complètent et s'articulent pour produire un code structuré, maintenable et extensible.

L'exercice 1.1 a illustré la puissance du polymorphisme et de l'abstraction dans un contexte de hiérarchie simple. L'exercice 1.2 a approfondi ces notions en ajoutant de nouvelles classes et méthodes, démontrant l'extensibilité offerte par l'héritage. Enfin, l'exercice 1.3 a montré comment l'encapsulation protège les données sensibles (comme le solde d'un compte) tout en offrant des opérations contrôlées.

Ces compétences constituent une base solide pour aborder des projets plus complexes utilisant des design patterns, des frameworks orientés objet, et les bonnes pratiques du développement logiciel professionnel.